

EXPERIMENT NO: 1

COMPARATIVE STUDY OF LOW-CODE DEVELOPMENT VERSUS TRADITIONAL APPLICATION DEVELOPMENT APPROACHES

1. AIM

To perform a comparative study of low-code application development and traditional application development approaches, to understand the stages involved in each methodology, and to analyse them against key criteria such as development speed, required skill level, flexibility, cost, scalability, maintenance and security, so as to determine which approach is best suited for a given category of software project.

2. INTRODUCTION TO THE TRADITIONAL APPROACH

Traditional application development, also referred to as conventional or code-centric development, is the classical method of building software in which the entire application is written manually using a programming language. Developers write the source code for the user interface, the business logic and the database layer, and then compile, test and deploy the resulting artefact. This approach normally follows a structured software development life cycle (SDLC) such as the Waterfall model, the Iterative model, the Spiral model or an Agile framework like Scrum.

In this approach, the development team consists of specialised roles including business analysts, software architects, front-end developers, back-end developers, database administrators, testers and DevOps engineers. Each role contributes to a specific phase of the life cycle. The application is typically built using languages such as Java, Python, C#, PHP or JavaScript, together with frameworks like Spring Boot, Django, ASP.NET or React, and databases such as MySQL, PostgreSQL, Oracle or MongoDB.

The principal strength of the traditional approach is complete control. Because every line of code is written by the development team, the architecture can be optimised for performance, the security model can be designed exactly as required, and any algorithm or third-party integration can be implemented without restriction. The organisation also owns the complete source code, which eliminates dependency on any external vendor and allows the software to be migrated, extended or licensed freely.

The main limitations are time and cost. Manual coding is slow, requires highly skilled engineers, and demands extensive testing before release. Any change in requirements involves modifying the source code, re-executing the test suite and redeploying the application, which increases the overall maintenance effort. For this reason, the traditional approach is most appropriate for complex, large-scale, performance-critical or highly regulated systems such as banking platforms, telecommunication systems, operating systems and real-time embedded software.

3. INTRODUCTION TO LOW-CODE DEVELOPMENT

Low-code development is a modern software development approach in which applications are created largely through a visual development environment rather than through hand-written source code. The developer assembles the application by dragging and dropping pre-built components onto a design canvas, configuring data models through forms, and defining business logic using visual workflow editors or simple expression languages. Hand-written code is used only where the built-in components are insufficient.

Low-code platforms provide a set of ready-made building blocks such as user interface controls, authentication modules, database connectors, reporting widgets and API integration adapters. The platform automatically generates the underlying code, provisions the database, manages the hosting environment and handles deployment. Widely used low-code platforms include Microsoft Power Apps, OutSystems, Mendix, Appian, Salesforce Lightning, ServiceNow App Engine and Zoho Creator.

A key characteristic of low-code development is that it enables citizen developers, that is, business analysts and domain experts who understand the process requirements but have limited programming expertise, to participate directly in building the application. This significantly shortens the gap between requirement definition and working software, and reduces the workload on professional development teams.

The primary advantages of low-code development are rapid delivery, reduced development cost, lower testing effort and simplified maintenance. The primary limitations are restricted customisation, dependence on the capabilities of the chosen platform, possible performance ceilings for very large workloads, and vendor lock-in, since migrating an application away from a low-code platform usually requires a complete rewrite. Low-code is therefore best suited to internal business applications, workflow automation, dashboards, approval systems and rapid prototypes.

4. DIAGRAM: STAGES IN THE TRADITIONAL APPROACH

The traditional approach follows the sequential phases of the software development life cycle. Each phase produces a defined deliverable that becomes the input for the next phase, and the process moves forward only after the previous stage is verified.

Stages in Traditional Application Development

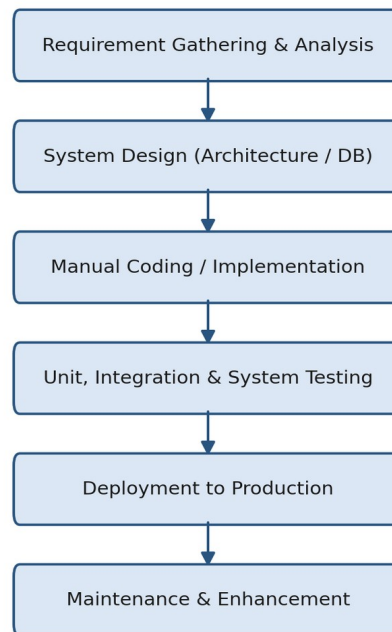


Fig. 1.1 – Stages in traditional application development

Requirement gathering captures the functional and non-functional needs of the client. System design converts these requirements into an architecture, database schema and interface design. Implementation is the manual coding phase. Testing verifies the software at unit, integration and system levels. Deployment moves the verified build into the production environment, and maintenance handles defect fixes and future enhancements.

5. DIAGRAM: STEPS IN LOW-CODE DEVELOPMENT

The low-code approach compresses the same objectives into a shorter, largely visual and highly iterative sequence of steps, in which several traditional phases are automated by the platform itself.

Steps in Low-Code Application Development

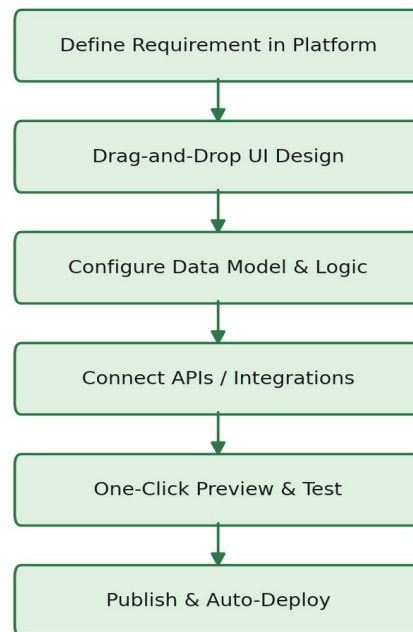


Fig. 1.2 – Steps in low-code application development

The requirement is defined directly inside the platform, the interface is assembled by dragging components onto a canvas, and the data model and business rules are configured through visual editors. External systems are connected using ready-made API connectors. The application can be previewed and tested instantly, and publishing is completed through an automated one-click deployment managed by the platform.

6. COMPARATIVE ANALYSIS

The following table presents a criteria-wise comparison between the traditional development approach and the low-code development approach.

Criteria	Traditional Development	Low-Code Development
Development Approach	Application is built by writing source code line by line using programming languages such as Java, Python, C# or JavaScript.	Application is built visually using drag-and-drop components, pre-built templates and configuration screens with minimal hand-written code.
Required Skill Level	Requires skilled programmers with strong knowledge of languages, frameworks, algorithms and databases.	Can be used by citizen developers, business analysts and domain experts with limited programming knowledge.
Development Speed	Slow. A medium-sized project may take several months because every layer is coded manually.	Fast. The same application can often be delivered in days or weeks because components are reusable.
Flexibility / Customisation	Very high. Any feature, algorithm or integration can be implemented without restriction.	Limited. Customisation is bounded by the features and extension points offered by the platform.
Cost of Development	High, due to large development teams, longer timelines and higher maintenance effort.	Lower initial cost, but recurring platform licensing or subscription charges apply.
Scalability	Highly scalable. Architecture can be tuned for very large user loads and complex workloads.	Suitable for small and medium workloads. Very large-scale systems may hit platform limits.
Testing Effort	Extensive manual and automated testing is required at unit, integration and system levels.	Reduced, because pre-built components are already tested by the platform vendor.
Deployment	Manual or CI/CD pipeline based deployment involving build servers, containers and environment setup.	One-click or automated publishing handled directly by the low-code platform.
Maintenance	Requires developers to modify source code, retest and redeploy for every change.	Changes are made through the visual interface, making updates quick and less error-prone.
Vendor Dependency	No dependency. The organisation fully owns the codebase and can migrate freely.	High dependency. Migration away from the platform is difficult and may require a rewrite.
Security Control	Complete control over authentication, encryption and security policies at code level.	Security is largely governed by the platform provider, with limited scope for deep customisation.
Best Suited	Complex enterprise systems, real-time	Internal business tools, dashboards,

Criteria	Traditional Development	Low-Code Development
For	applications, operating systems and performance-critical software.	workflow automation, CRUD applications and rapid prototypes.

7. RESULT

The comparative study of low-code development and traditional application development approaches was carried out successfully. The stages of the traditional software development life cycle and the steps of the low-code development process were studied and represented diagrammatically, and both approaches were analysed against twelve evaluation criteria.

The study concludes that neither approach is universally superior; each is suited to a different class of problem. The traditional approach offers complete control, unrestricted customisation, high scalability and full ownership of the source code, making it the correct choice for complex, performance-critical and highly secure enterprise systems, at the cost of longer development time, higher expenditure and the need for skilled programmers. The low-code approach offers substantially faster delivery, lower development cost, reduced testing effort and easier maintenance, making it the correct choice for internal business applications, workflow automation and rapid prototyping, at the cost of limited customisation, platform dependency and vendor lock-in.

In current industry practice, many organisations adopt a hybrid strategy in which low-code platforms are used to deliver standard business applications quickly, while traditional coding is reserved for the core, complex and performance-sensitive components of the system. Thus, the aim of the experiment has been achieved.